

BPF 与 DKapture 技术介绍

Content

目录

01

BPF架构概览

02

BPF工作流程

03

BPF数据交互

04

DKapture项目介绍

05

实战-追踪malloc操作的实现

06

总结

01

BPF架构概览

1. 什么是BPF
2. 架构图
3. BPF的分类和使用场景



什么是BPF?

eBPF (extended Berkeley Packet Filter) 是 Linux 内核中的一个革命性技术，它允许在内核空间运行沙箱程序，而无需修改内核源代码或加载内核模块。

● eBPF 的核心特性

安全性：通过验证器确保程序不会导致内核崩溃

高性能：JIT 编译器将 eBPF 字节码编译为本地机器码

可观测性：可以在任意内核/用户空间位置插入探针

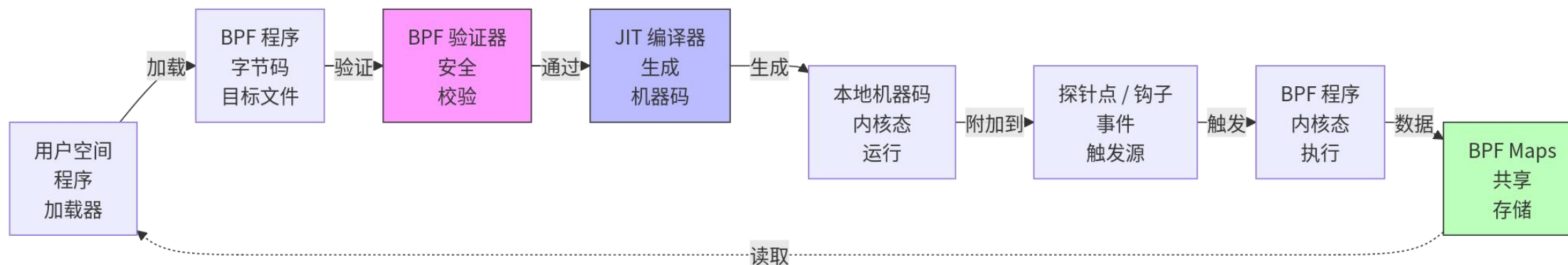
灵活性：支持网络、安全、性能分析等多种场景

01 BPF架构概览

■ 以用为本 拥抱创新

架构图

用户态程序把 BPF 字节码加载进内核，验证器确认安全后由 JIT 编译为本地机器码；程序附加到探针/钩子，事件触发后在内核态执行，并通过 BPF Maps 与用户态交换数据。



BPF程序类型

eBPF 支持 30+ 种程序类型，每种类型针对不同的使用场景。

程序类型	常量	附加点	主要用途
Kprobe/Uprobe	BPF_PROG_TYPE_KPROBE	内核/用户函数	动态追踪
Tracepoint	BPF_PROG_TYPE_TRACEPOINT	静态追踪点	稳定追踪
XDP	BPF_PROG_TYPE_XDP	网卡驱动	高性能包处理
TC	BPF_PROG_TYPE_SCHEDCLS	网络队列	流量控制
LSM	BPF_PROG_TYPE_LSM	安全模块	安全策略

02

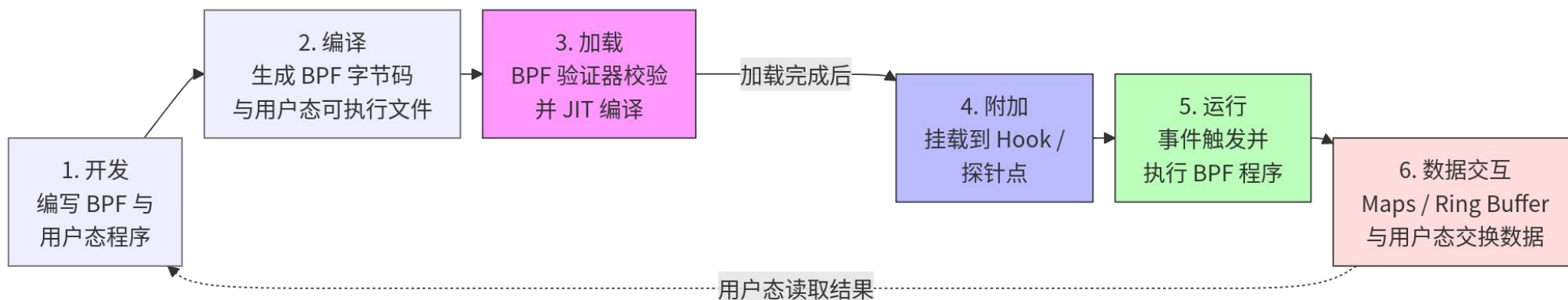
BPF工作流程

- 1.完整工作流程
- 2.开发阶段
- 3.编译阶段
- 4.加载阶段
- 5.共享机制的底层实现



完整工作流程

先编写并编译 BPF 程序，加载到内核完成验证与 JIT 编译，再附加到目标 Hook/探针；事件触发后执行 BPF 程序，并通过 Maps / Ring Buffer 与用户态交换数据，形成完整的开发—运行—数据交互闭环。

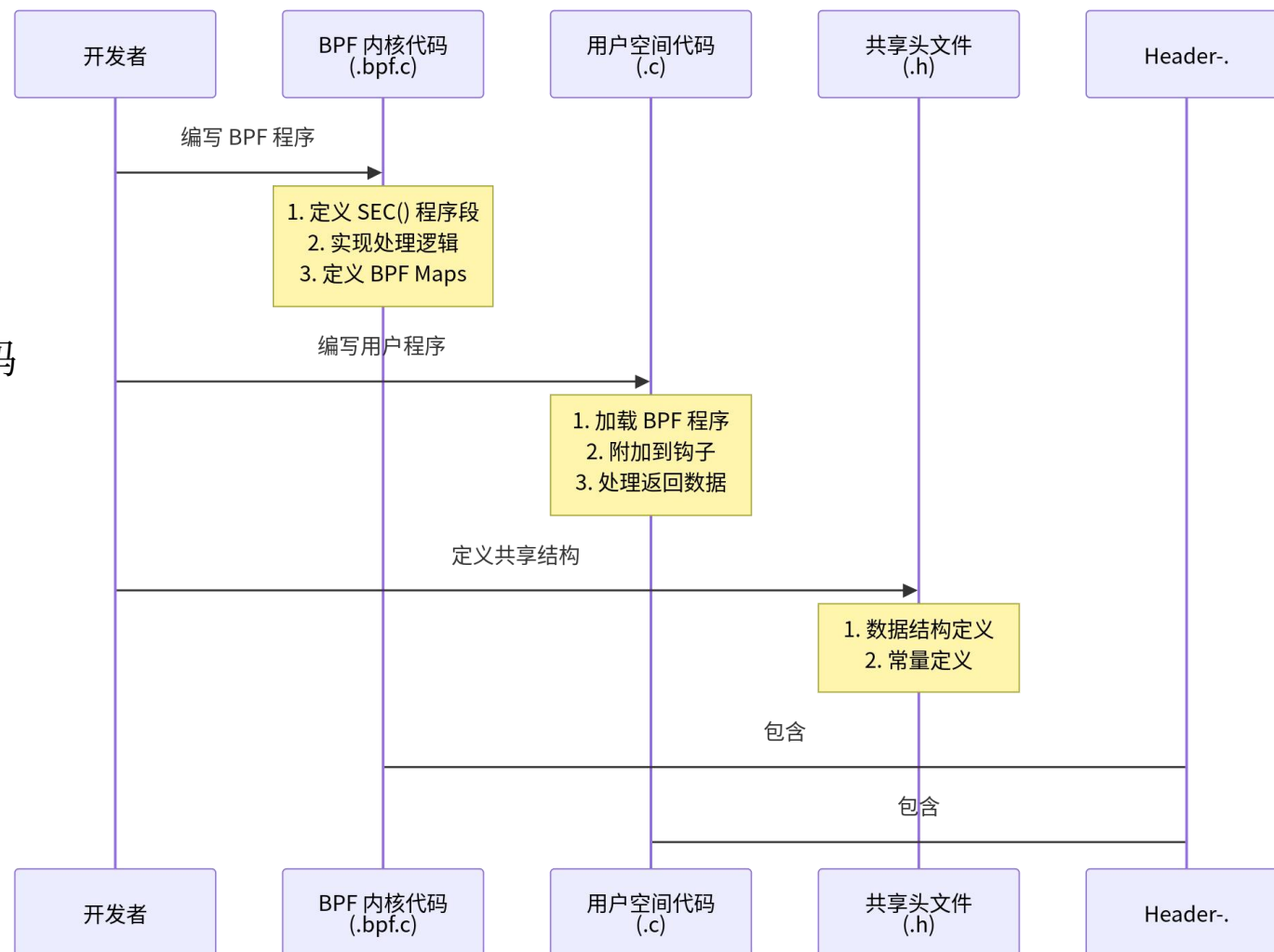


02 BPF工作流程

■ 以用为本 拥抱创新

开发阶段

```
myproject/  
├── src/  
│   ├── myprogram.bpf.c # BPF 内核代码  
│   └── myprogram.c # 用户空间代码  
├── include/  
│   └── common.h # 共享定义  
└── Makefile
```



02 BPF工作流程

以用为本 拥抱创新

编译阶段

1. 编译 BPF 程序

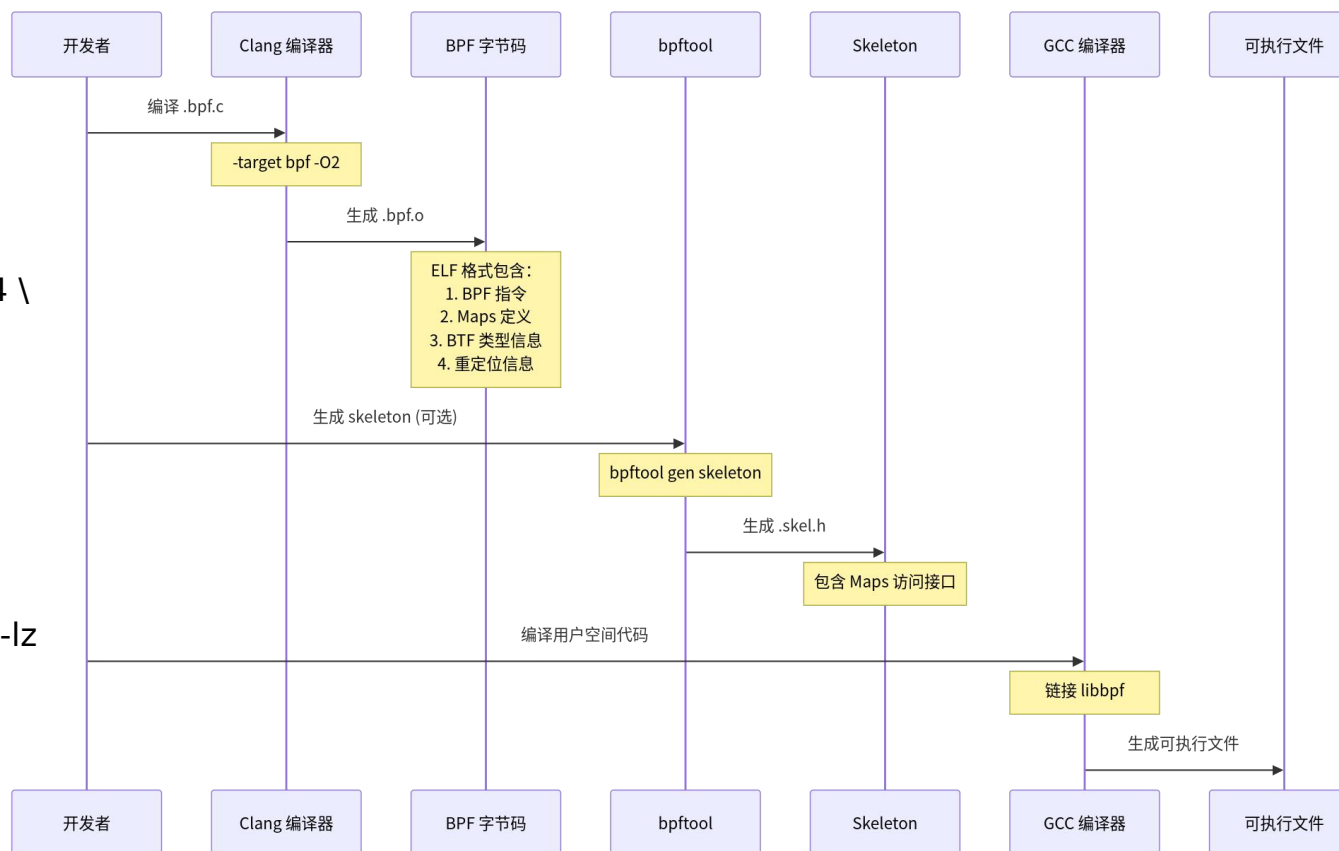
```
clang -g -O2 -target bpf -D__TARGET_ARCH_x86_64 \  
-I/usr/include/x86_64-linux-gnu \  
-c myprogram.bpf.c -o myprogram.bpf.o
```

2. 生成 skeleton (可选, 推荐)

```
bpftool gen skeleton myprogram.bpf.o  
> myprogram.skel.h
```

3. 编译用户空间程序

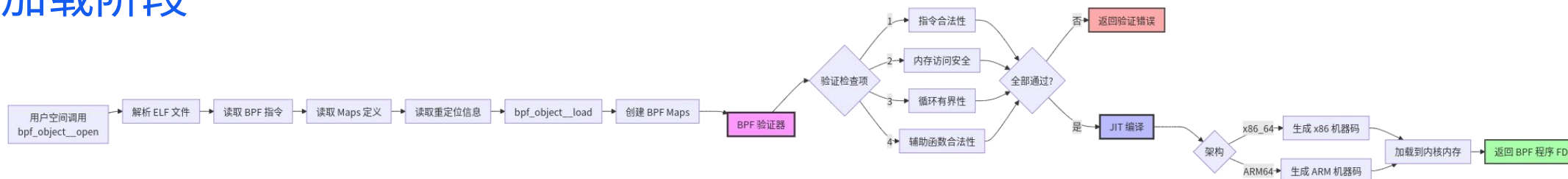
```
gcc -g -Wall -o myprogram myprogram.c -lbpf -lelf -lz
```



02 BPF工作流程

以用为本 拥抱创新

加载阶段



检查项	说明	失败原因示例
指令合法性	检查是否使用了合法的 BPF 指令	使用了未定义的指令码
内存访问	所有内存访问必须经过验证	访问越界、访问无效指针
循环边界	循环必须有明确的终止条件	无界循环、递归
辅助函数	只能调用允许的辅助函数	调用未授权的函数
类型安全	BTF 类型检查	类型不匹配
栈大小	栈使用不能超过 512 字节	栈溢出
指令数量	程序复杂度限制	超过 100 万条指令

map共享机制

Pin 注册表文件

文件路径: /var/tmp/.dkapture-pin-registry

作用: 集中管理所有 DKapture 创建的 BPF Map Pin 路径, 实现跨进程的 Map 发现和共享。

```
1 {
2   "maps": {
3     "events": {
4       "pin_path": "/sys/fs/bpf/dkapture_events",
5       "type": "BPF_MAP_TYPE_RINGBUF",
6       "max_entries": 262144,
7       "created_at": 1726396800,
8       "pid": 12345,
9       "program": "dkapture-tracer"
10    },
11    "malloc_stats": {
12      "pin_path": "/sys/fs/bpf/dkapture_malloc_stats",
13      "type": "BPF_MAP_TYPE_HASH",
14      "key_size": 4,
15      "value_size": 8,
16      "max_entries": 10240,
17      "created_at": 1726396805,
18      "pid": 12345,
19      "program": "dkapture-tracer"
20    }
21  },
22  "version": "1.0",
23  "last_updated": 1726396805
24 }
```

字段	说明
pin_path	Map 固定在 BPF 文件系统 中的路径
type	Map 类型 (RINGBUF、HASH、ARRAY 等)
max_entries	Map 最大条目数
key_size / value_size	键值大小 (仅 Hash/Array 类型)
created_at	创建时间戳 (Unix 时间)
pid	创建进程的 PID
program	创建程序名称

02 BPF工作流程

■ 以用为本 拥抱创新

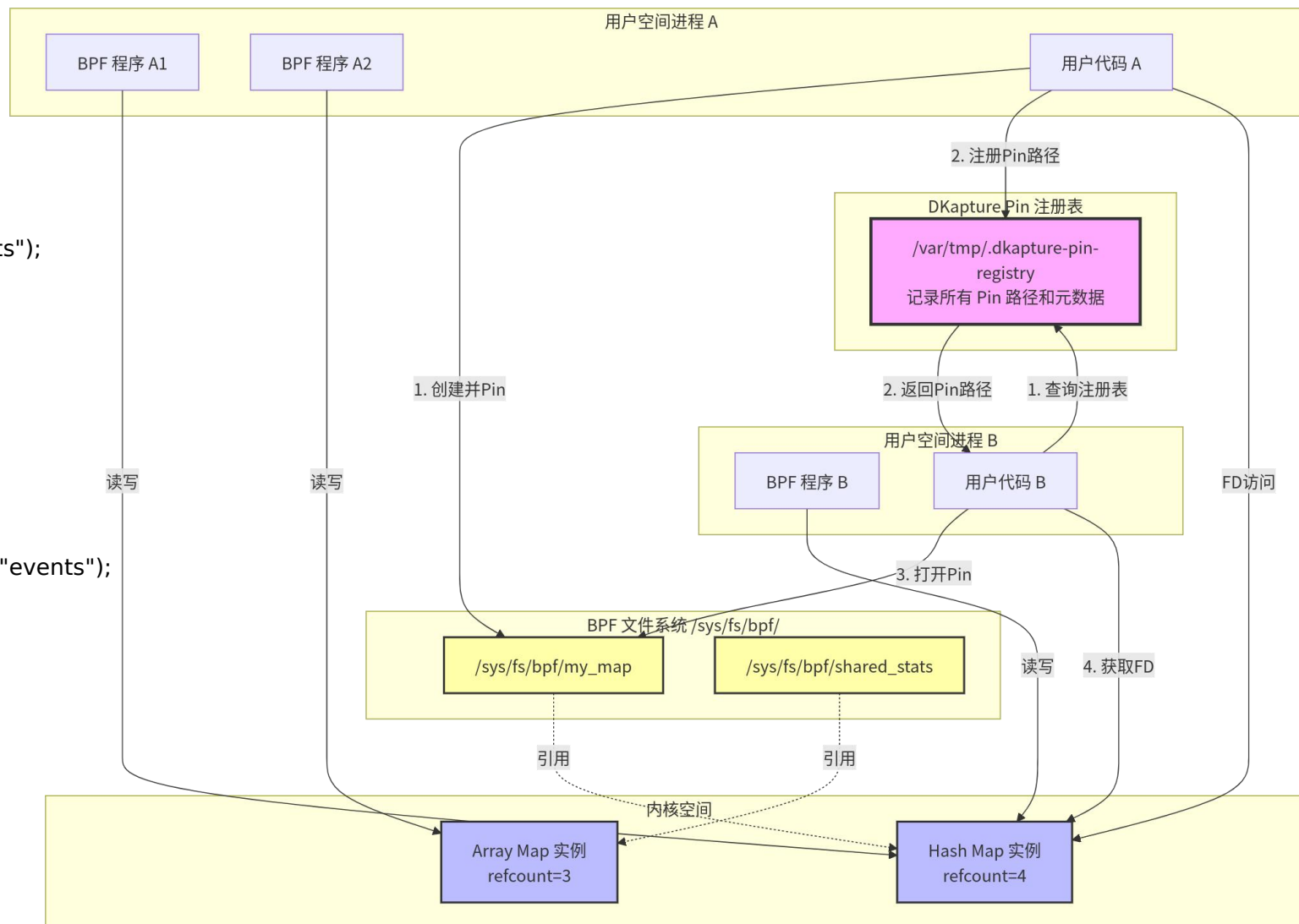
map共享机制

进程A创建map

```
// 1. 创建并 Pin Map  
bpf_map__pin(map, "/sys/fs/bpf/dkapture_events");  
  
// 2. 注册到 Pin Registry  
register_pin_to_registry("events",  
"/sys/fs/bpf/dkapture_events", map_info);
```

进程B查找并重用map

```
// 1. 从 Registry 查询 Pin 路径  
const char *pin_path = lookup_pin_from_registry("events");  
// 返回: "/sys/fs/bpf/dkapture_events"  
  
// 2. 打开已存在的 Map  
int map_fd = bpf_obj_get(pin_path);  
  
// 3. 使用共享 Map  
bpf_map_lookup_elem(map_fd, &key, &value);
```



map共享机制

extern 关键字与 ELF 改造

背景问题

多个 BPF 程序 / 多个进程需要共享同一个 Map 实例。但存在两个障碍：

bpftool gen skeleton 和 libbpf 原生不支持 extern SEC(".maps") 声明，遇到会报错

libbpf 的 bpf_object__load() 默认为每个 Map 调用 bpf_create_map 创建独立实例，需要机制来复用已有 Map

解决方案

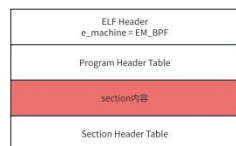
当使用 extern 关键字声明共享 Map 时，编译器会对 ELF 文件进行特殊处理，实现多个 BPF 程序共享同一个 Map。

```
1 // 程序 A: 定义 Map
2 struct {
3     __uint(type, BPF_MAP_TYPE_RINGBUF);
4     __uint(max_entries, 262144);
5 } events SEC(".maps");
6
7 // 程序 B: 使用 extern 引用 Map
8 extern struct {
9     __uint(type, BPF_MAP_TYPE_RINGBUF);
10    __uint(max_entries, 262144);
11 } events SEC(".maps"); // ← extern 关键字
```

02 BPF 工作流程

以用为本 拥抱创新

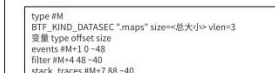
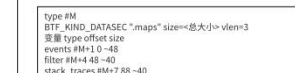
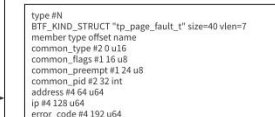
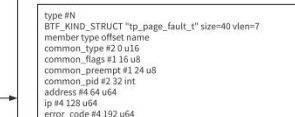
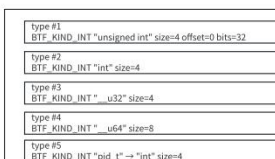
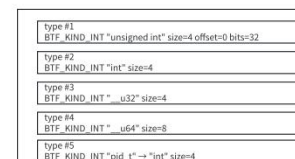
map共享机制



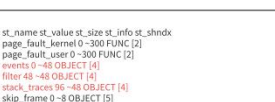
编译期 ELF 手术



添加零填充



修改 st_value, st_info, st_shndx



"events[0]filter[0]stack_traces[0]"

写入

写入

写入

添加

map 共享机制

进程 A：创建并固定 Map

```
// 加载 BPF 程序
obj = bpf_object__open_file("program.bpf.o", NULL);
bpf_object__load(obj);

// 获取 Map
map = bpf_object__find_map_by_name(obj, "shared_map");
map_fd = bpf_map__fd(map);

// 固定 Map 到文件系统（关键步骤！）
const char *pin_path = "/sys/fs/bpf/shared_map";
int err = bpf_map__pin(map, pin_path);

// 写入数据
__u32 key = 123;
__u64 value = 456;
bpf_map_update_elem(map_fd, &key, &value, BPF_ANY);
```

进程 B：重用已固定的 Map

```
// 从文件系统打开已存在的 Map（关键步骤！）
map_fd = bpf_obj_get("/sys/fs/bpf/shared_map");

// 读取进程 A 写入的数据
__u32 key = 123;
__u64 value;

if (bpf_map_lookup_elem(map_fd, &key, &value)
    == 0) {
    printf("Read from shared map: key=%u,
value=%llu\n", key, value);
}
```

03

BPF数据交互

- 1.统计聚合方式
- 2.事件流方式



两种数据交互的特点

Hash Map统计+用户态聚合

使用场景

- 统计型数据（计数、求和、平均值）
- 需要按键分组聚合
- 数据更新频繁但不需要实时流式输出
- 适合周期性轮询读取

典型应用

- 进程级别的系统调用计数
- 函数调用频率统计
- 网络连接统计
- 资源使用量统计

Ring Buffer 输出+用户态读取

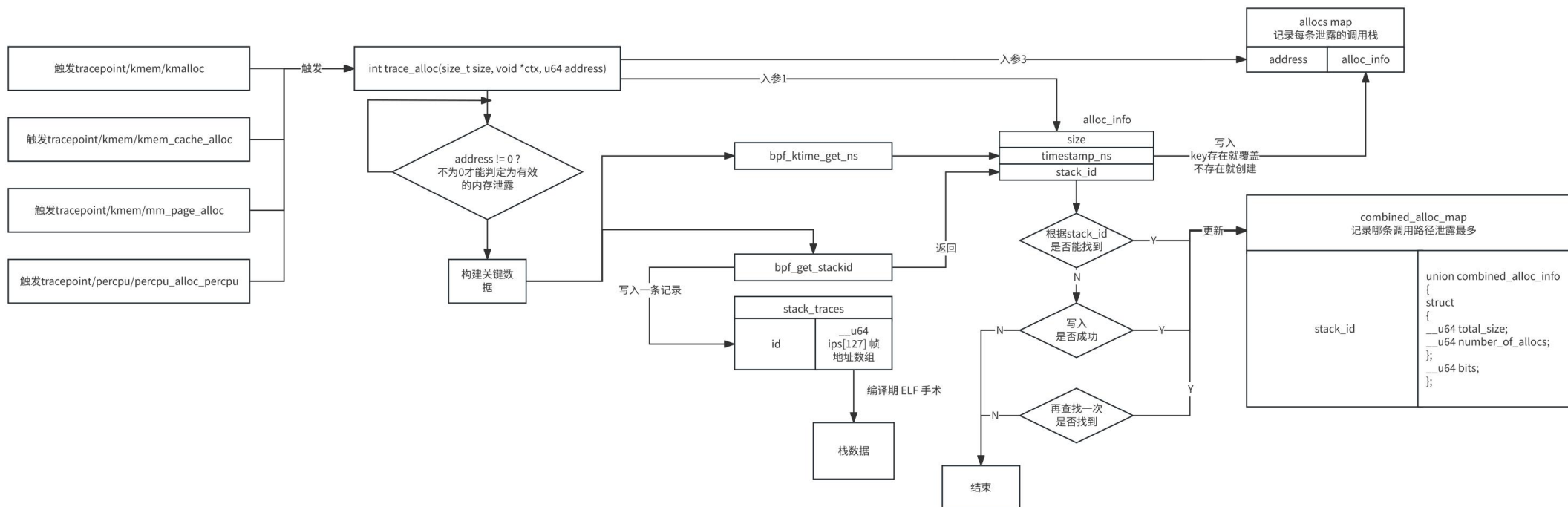
使用场景

- 事件流数据（需要保留每次事件的详细信息）
- 实时性要求高
- 需要记录事件发生的时间序列
- 数据量大但不需要聚合

典型应用

- 详细的函数调用追踪
- 安全审计日志
- 网络包捕获
- 性能事件记录

Hash Map统计+用户态聚合



BPF 程序端（内核空间）

每次事件发生时，通过 `bpf_map_lookup_elem()` 查找 key
如果存在，原子更新计数器（`__sync_fetch_and_add`）
如果不存在，初始化为 1

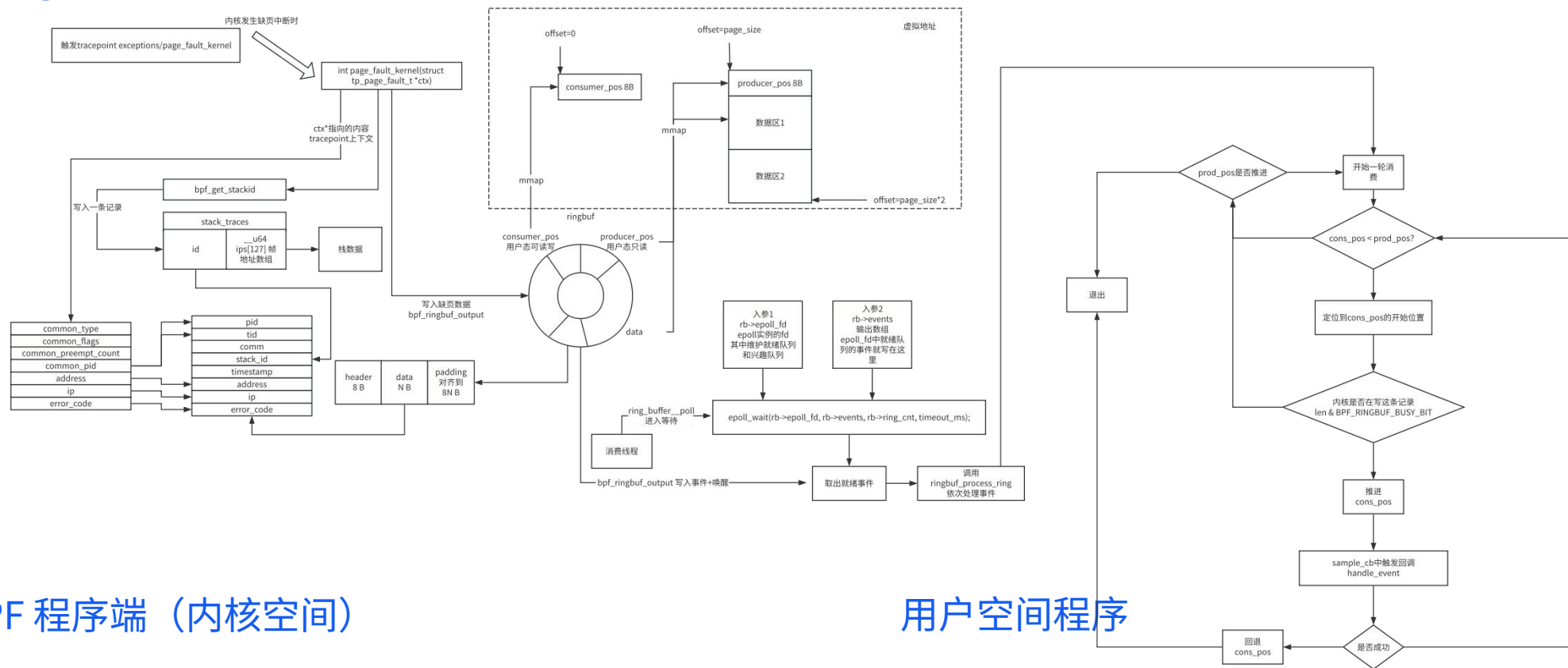
用户空间程序

使用 `bpf_map_get_next_key()` 遍历所有 key
使用 `bpf_map_lookup_elem()` 读取每个 key 的 value
聚合、排序、格式化输出

03 BPF数据交互

以用为本 拥抱创新

Ring Buffer输出+用户态读取



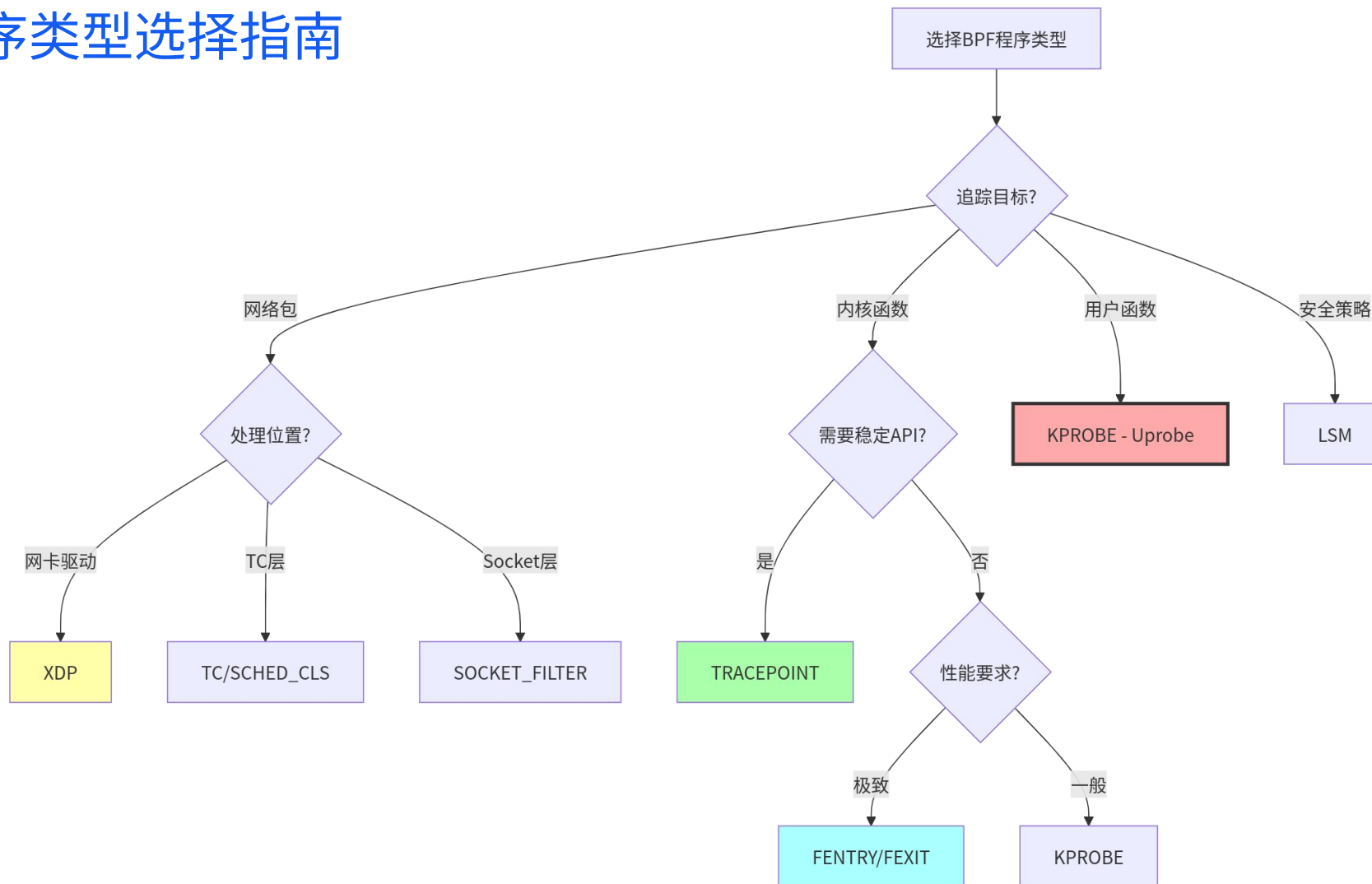
BPF 程序端 (内核空间)

使用 `bpf_ringbuf_reserve()` 预留空间
填充事件数据结构
使用 `bpf_ringbuf_submit()` 提交事件

用户空间程序

使用 `ring_buffer_new()` 创建 Ring Buffer 实例
注册回调函数处理每个事件
使用 `ring_buffer_poll()` 轮询并触发回调

BPF程序类型选择指南



04

DKapture项目介绍

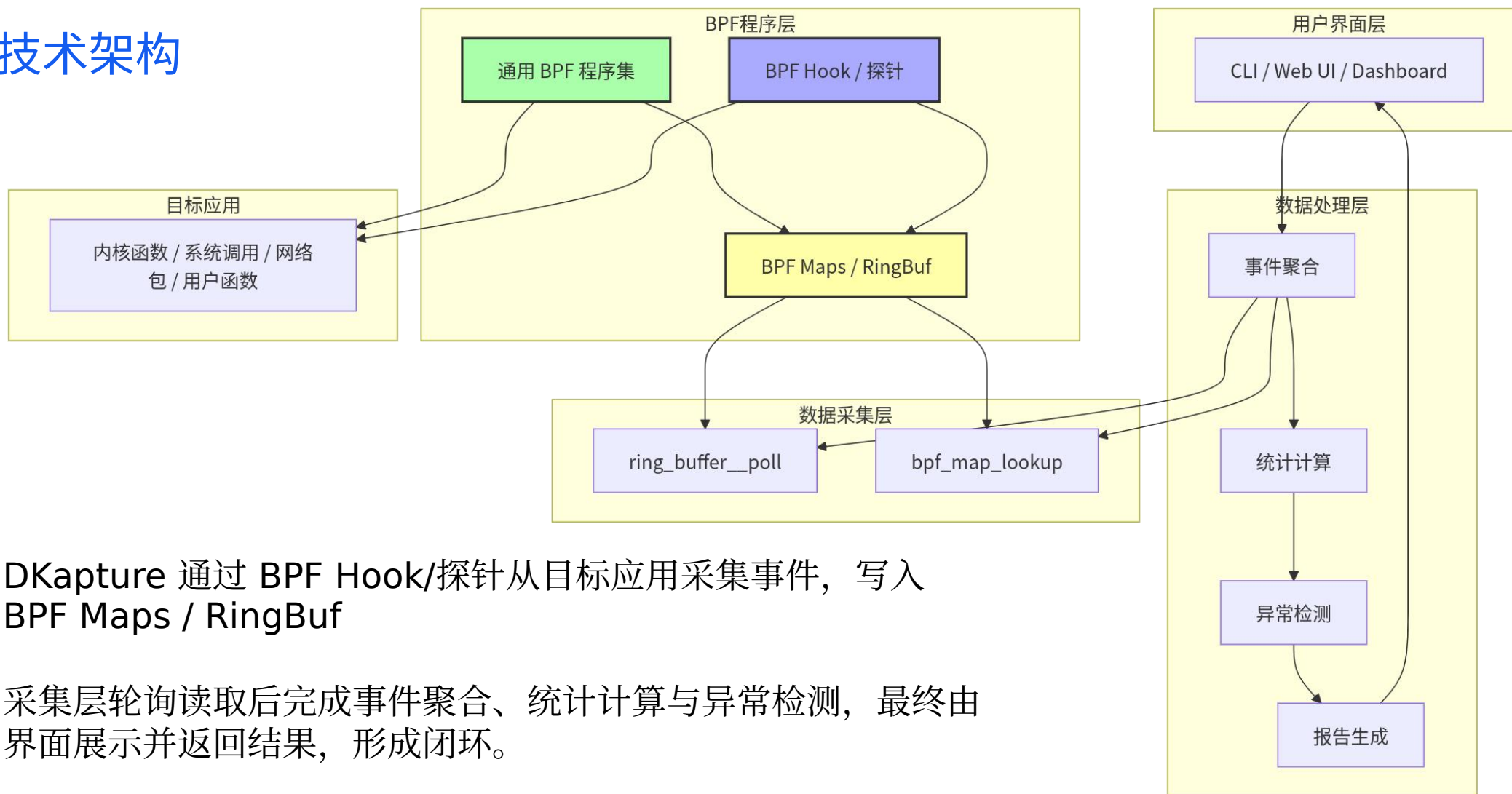
- 1.技术架构
- 2.几个应用场景
- 3.项目优势



04 DKapture项目介绍

■ 以用为本 拥抱创新

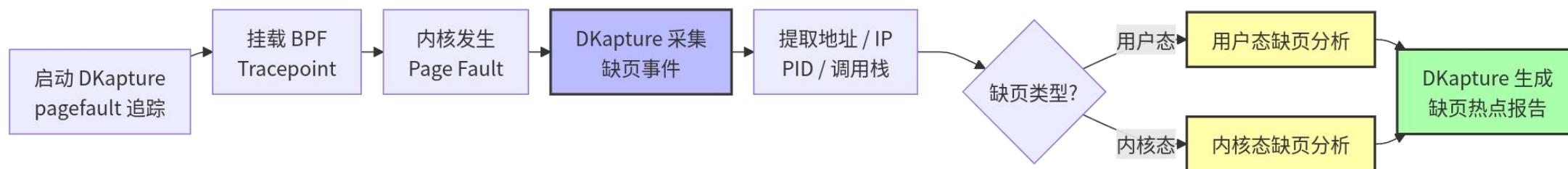
技术架构



04 DKapture项目介绍

■ 以用为本 拥抱创新

应用场景1:缺页异常追踪



内核态探针: `~/Desktop/tech/dkapture-bpf/observe/pagefault.bpf.c`

用户态加载与读取: `~/Desktop/tech/libdkapture/observe/pagefault.cpp`

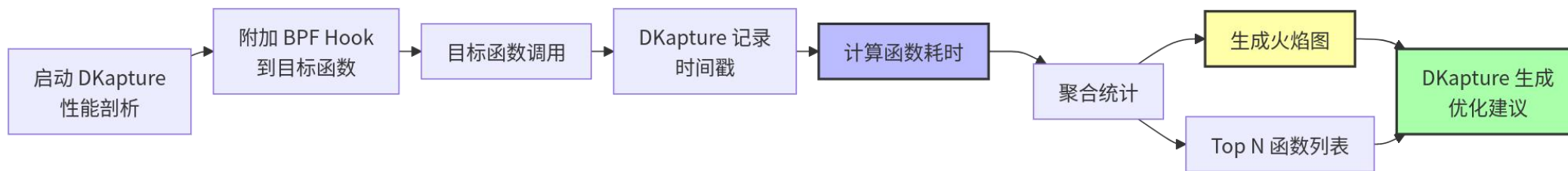
挂载点: `tracepoint/exceptions/page_fault_kernel`、`tracepoint/exceptions/page_fault_user`

数据通道: `events RingBuf`

进程过滤: `filter Array Map`

调用栈: `stack_traces Stack Trace Map`

应用场景2:性能热点分析



内核态探针: `~/Desktop/tech/dkapture-bpf/observe/sched-snoop.bpf.c`

用户态读取: `~/Desktop/tech/libdkapture/observe/sched-snoop.cpp`

挂载点: `tp/sched/sched_switch`、`tp/sched/sched_stat_runtime`、`tp/sched/sched_stat_wait`

数据通道: `logs RingBuf`

过滤配置: `filter Hash Map`

04 DKapture项目介绍

以用为本 拥抱创新

应用场景3:SSL密钥获取

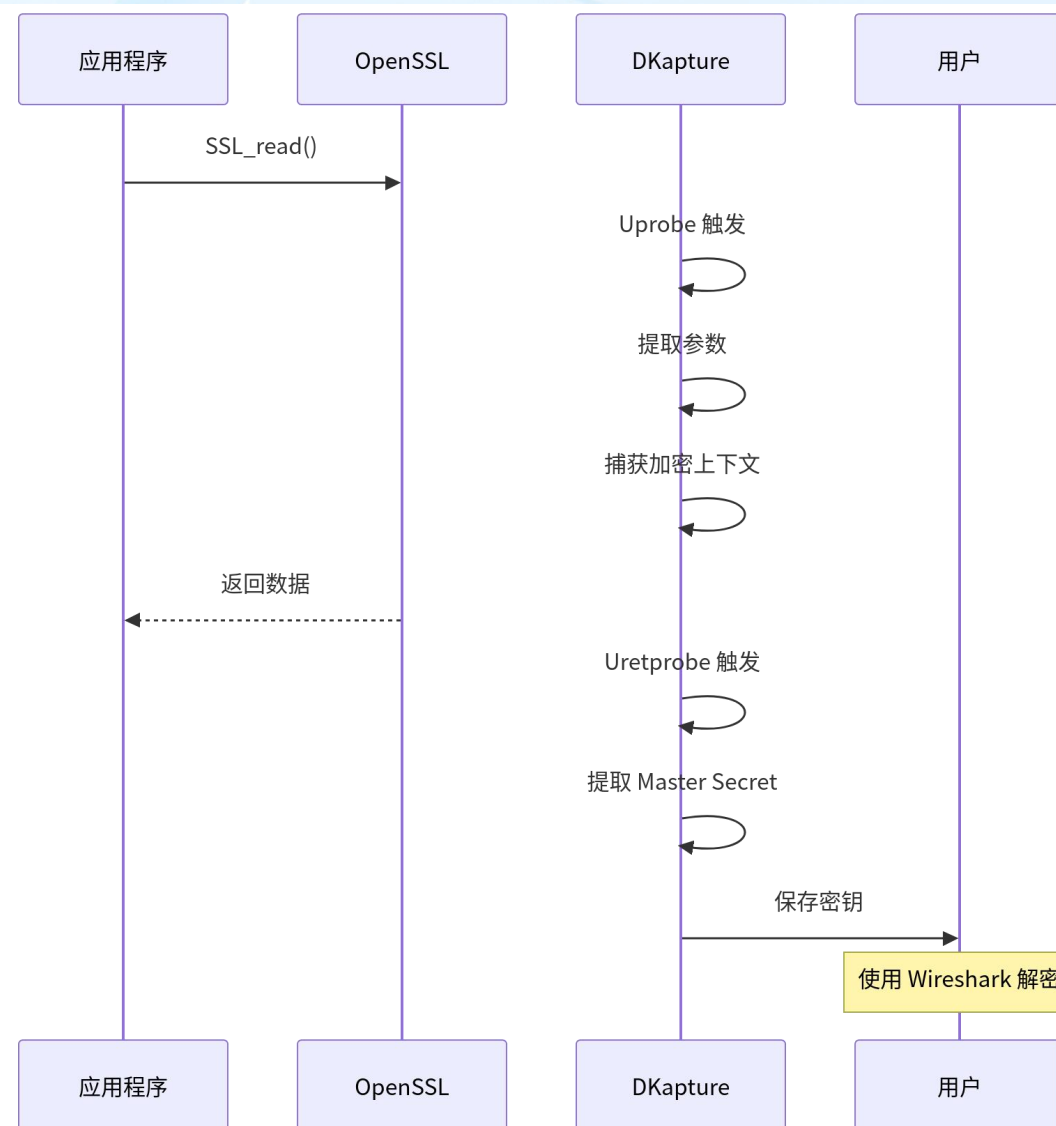
探针类型: uprobe / uretprobe

推荐挂载点: uprobe/SSL_read、
uretprobe/SSL_read (或 OpenSSL 对应符号
地址)

数据通道: events RingBuf

关键字段: 进程 PID/TID、调用参数、返回值、
加密上下文 / Master Secret

用户态处理: ring_buffer_poll 回调保存密钥



DKapture vs 传统工具对比

维度	DKapture	Valgrind	GDB	SystemTap
侵入性	✓零侵入	⚠中等	✗高	✓零侵入
性能开销	✓1-5%	✗20-50x	✗停止运行	✓2-10%
生产可用	✓是	✗否	✗否	⚠受限
实时性	✓实时	✗离线	✗交互	✓实时
学习曲线	✓简单	⚠中等	⚠中等	✗陡峭
内核要求	≥5.5	无	无	自定义

DKapture的核心优势

优势项	优势原因
更安全	基于 eBPF + 内核验证器，避免 sysdig/systemtap 的内核模块风险
更低开销	内核态通过各种map完成过滤与聚合，ringbuf 批量输出，减少用户态事件处理
更快 进程信息读取比 procfs 快一个数量级以上	传统方式要逐个打开、读取、关闭每个 /proc/[pid]/stat 文件。DKapture 使用 BPF 在内核态直接采集进程信息，用户态只需要少量 read 和 BPF 交互，避免海量文件系统调用。
更高吞吐 文件跟踪 324 事件/秒，高于 sysdig 249、systemtap 170	内核态直接 hook 文件系统相关事件。 内核态先过滤和聚合，减少无效事件进入用户态。 现代 RINGBUF 输出路径比传统工具的用户态消费方式更高效。
能力更全	observe 观测、filter 过滤、policy 策略一体化
更现代 覆盖更多现代 eBPF 挂载点	支持 fentry/fexit、tp_btf、LSM、XDP、BTF/CO-RE、skeleton
更易共享	pin-registry + bpf-linker 实现多进程共享 BPF Map

05

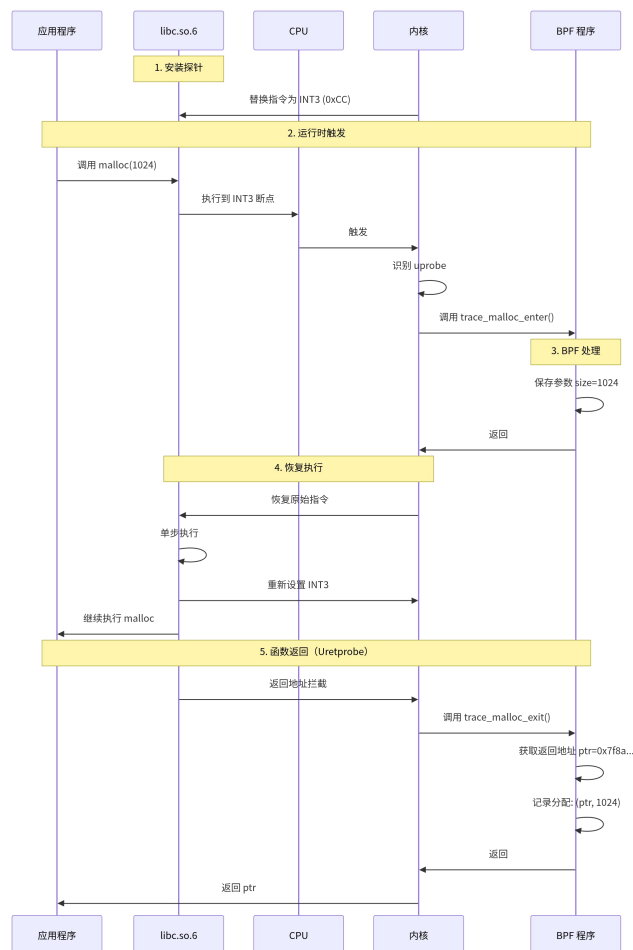
实战-追踪malloc操作的实现

1. 探针选取和原理
2. 数据流转设计
3. malloc符号追踪
4. 架构设计
5. 代码解读

05 实战-追踪malloc操作的实现

以用为本 拥抱创新

探针选取和原理-Uprobe和Uretprobe协作完成



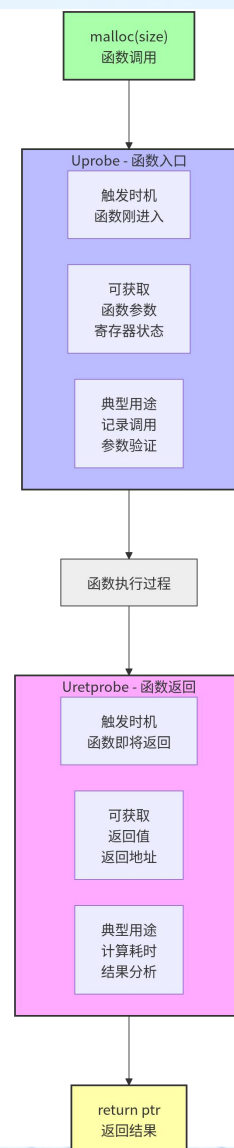
为什么需要两者配合？

对于 `malloc(size)` 这样的函数，我们需要追踪：

输入参数：size（分配多少内存？）

返回结果：ptr（分配到哪个地址？）

但是，单独使用 Uprobe 或 Uretprobe 都无法同时获取这两个信息！

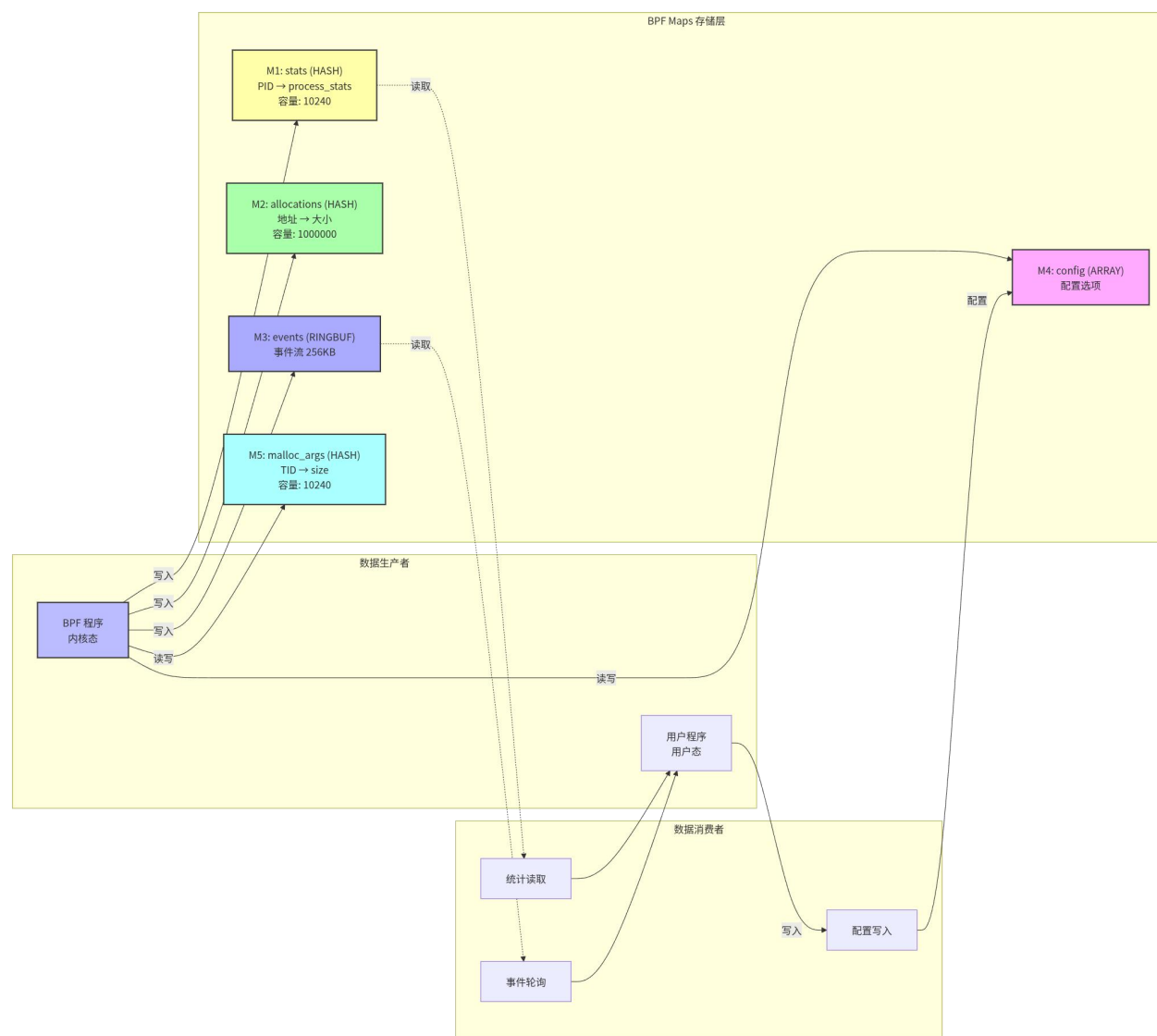


05 实战-追踪malloc操作的实现

以用为本 拥抱创新

数据流转设计-四个HashMap+一个RingBuffer+一个Array

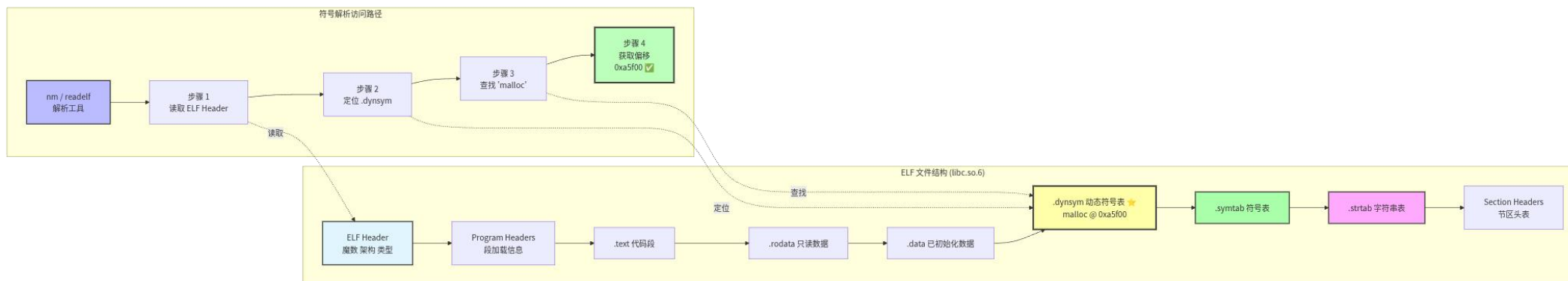
Map 名称	类型	作用
stats	HASH	存储每个进程的统计信息（分配/释放次数、字节数）
allocations	HASH	记录地址→大小映射，用于 free 时查找释放的大小
events	RINGBUF	传输详细事件到用户空间（详细模式）
config	ARRAY	配置选项（是否启用详细输出）
malloc_args	HASH	临时存储 malloc 参数（uprobe 与 uretprobe 间传递）



05 实战-追踪malloc操作的实现

■ 以用为本 拥抱创新

malloc符号追踪



```
nm -D /lib/x86_64-linux-gnu/libc.so.6 |  
grep " malloc$"
```

```
readelf -s /lib/x86_64-linux-gnu/libc.so.6 |  
grep malloc
```

```
static size_t get_symbol_offset_simple(const char *lib, const char *symbol)
{
    char cmd[512];
    FILE *fp;
    size_t offset = 0;

    snprintf(cmd, sizeof(cmd),
             "nm -D %s 2>/dev/null | grep ' %s$' | awk '{print $1}'",
             lib, symbol);

    fp = popen(cmd, "r");
    if (fp) {
        if (fscanf(fp, "%lx", &offset) != 1) {
            offset = 0;
        }
        pclose(fp);
    }

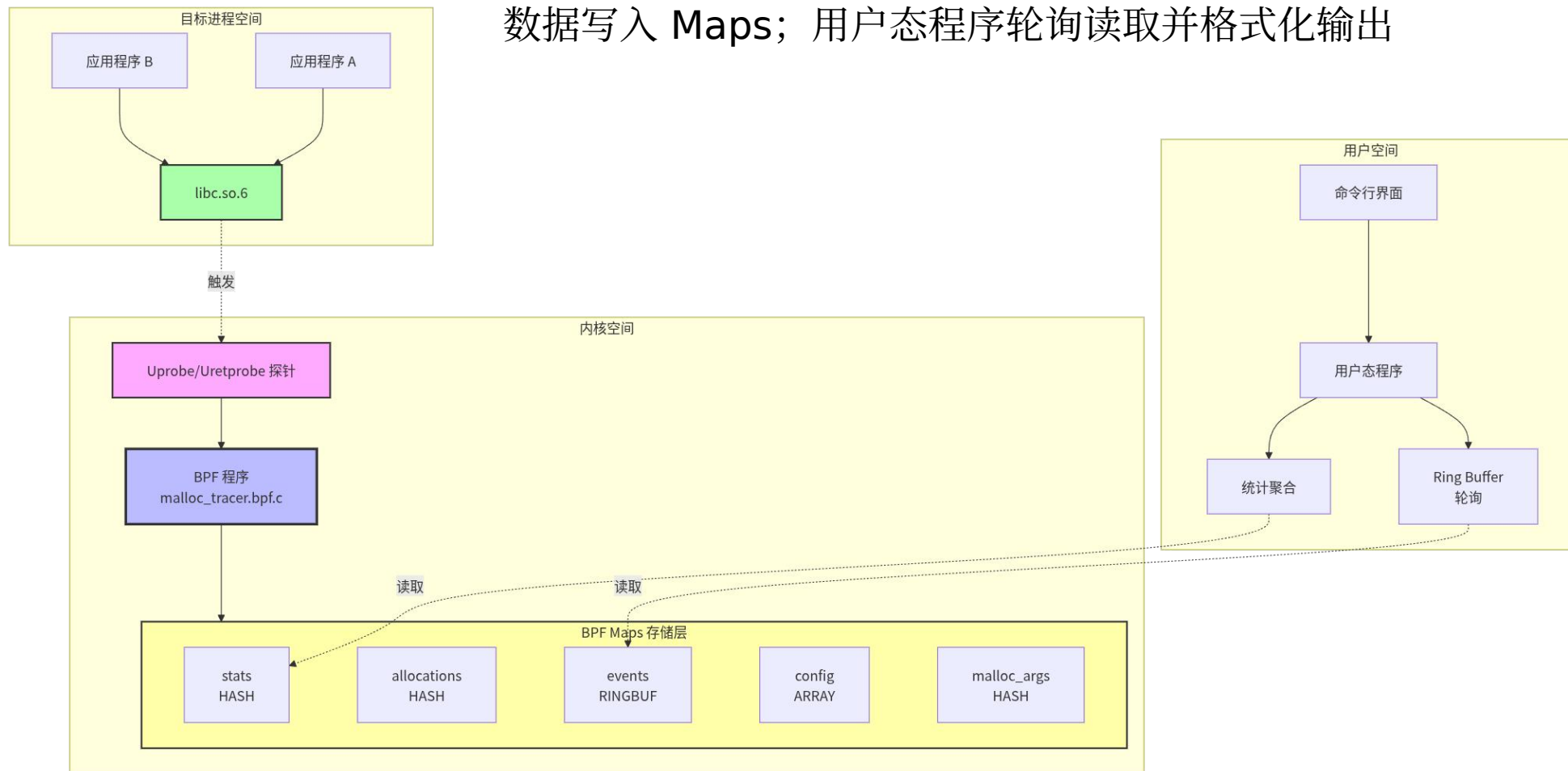
    return offset;
}
```

05 实战-追踪malloc操作的实现

■ 以用为本 拥抱创新

架构设计-逻辑层面

用户程序加载 BPF 程序并附加到 libc 的 malloc/free/calloc 函数；目标进程调用这些函数时触发探针，BPF 程序在内核态收集数据写入 Maps；用户态程序轮询读取并格式化输出

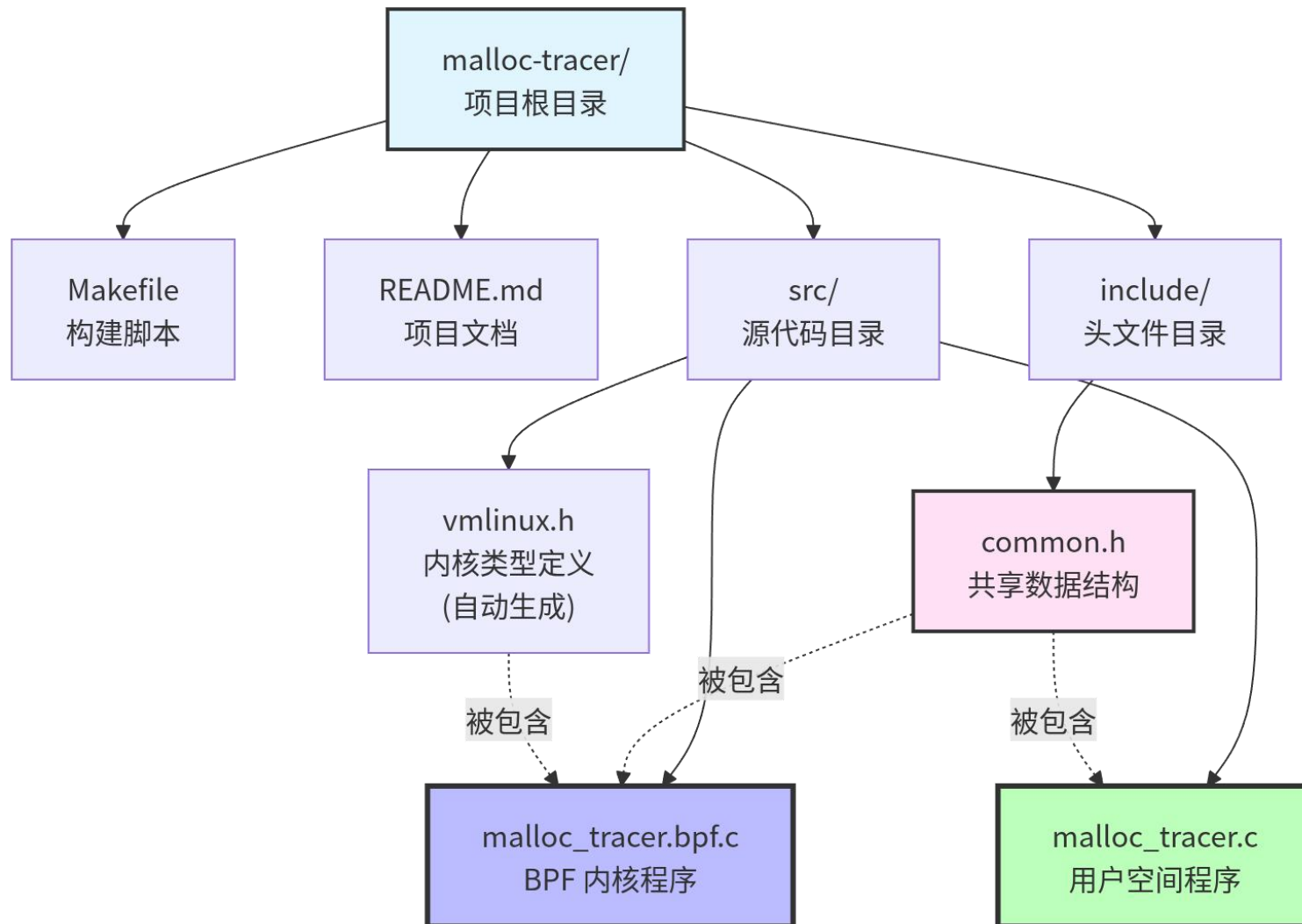


05 实战-追踪malloc操作的实现

■ 以用为本 拥抱创新

架构设计-文件

文件	作用	编译产物
common.h	定义共享数据结构 (alloc_event, process_stats)	-
malloc_tracer. bpf.c	BPF 内核程序, 定义 Maps 和 探针处理函数	malloc_tracer. bpf.o
malloc_tracer. c	用户空间程序, 加载 BPF、附 加探针、读取 数据	malloc_tracer
vmlinux.h	内核类型定义 (bpftool 自动 生成)	-



05 实战-追踪malloc操作的实现

关键代码-malloc追踪

```
SEC("uprobe/malloc")
int BPF_KPROBE(trace_malloc_enter, size_t size)
{
    __u64 pid_tgid = bpf_get_current_pid_tgid();
    __u32 tid = (__u32)pid_tgid;

    // 保存 size 参数
    bpf_map_update_elem(&malloc_args, &tid, &size, BPF_ANY);

    return 0;
}
```

```
SEC("uretprobe/malloc")
int BPF_KRETPROBE(trace_malloc_exit, void *ret_ptr)
{
    __u64 pid_tgid = bpf_get_current_pid_tgid();
    __u32 pid = pid_tgid >> 32;
    __u32 tid = (__u32)pid_tgid;

    // 获取保存的 size 参数
    __u64 *size_ptr = bpf_map_lookup_elem(&malloc_args, &tid);
    if (!size_ptr)
        return 0;

    __u64 size = *size_ptr;
    bpf_map_delete_elem(&malloc_args, &tid);

    if (!ret_ptr)
        return 0; // 分配失败

    __u64 addr = (__u64)ret_ptr;

    // 记录分配
    bpf_map_update_elem(&allocations, &addr, &size, BPF_ANY);

    // 更新统计
    update_stats(pid, size, 1);

    // 发送事件
    send_event(EVENT_MALLOC, addr, size);

    return 0;
}
```

```
SEC("uprobe/free")
int BPF_KPROBE(trace_free_enter, void *ptr)
{
    if (!ptr)
        return 0; // free(NULL) 是合法的

    __u64 pid_tgid = bpf_get_current_pid_tgid();
    __u32 pid = pid_tgid >> 32;
    __u64 addr = (__u64)ptr;

    // 查找分配大小
    __u64 *size_ptr = bpf_map_lookup_elem(&allocations, &addr);
    __u64 size = size_ptr ? *size_ptr : 0;

    // 删除分配记录
    bpf_map_delete_elem(&allocations, &addr);

    // 更新统计 (size 为负)
    if (size > 0) {
        update_stats(pid, -(__s64)size, 0);
    }

    // 发送事件
    send_event(EVENT_FREE, addr, size);

    return 0;
}
```

```
static __always_inline void update_stats(__u32 pid, __s64 size_delta, int is_alloc)
{
    struct process_stats *st;
    struct process_stats zero = {};

    // 查找或创建统计记录
    st = bpf_map_lookup_elem(&stats, &pid);
    if (!st) {
        bpf_map_update_elem(&stats, &pid, &zero, BPF_ANY);
        st = bpf_map_lookup_elem(&stats, &pid);
        if (!st)
            return;
    }

    // 更新计数器
    if (is_alloc) {
        __sync_fetch_and_add(&st->total_allocs, 1);
        __sync_fetch_and_add(&st->total_bytes, size_delta);
        __sync_fetch_and_add(&st->current_bytes, size_delta);

        // 更新峰值
        __u64 current = st->current_bytes;
        if (current > st->peak_bytes) {
            st->peak_bytes = current;
        }
    } else {
        __sync_fetch_and_add(&st->total_frees, 1);
        // size_delta 为负数
        __u64 current = st->current_bytes;
        if (current >= ((__u64)(-size_delta))) {
            __sync_fetch_and_add(&st->current_bytes, size_delta);
        }
    }
}
```

```
static __always_inline void send_event(__u8 type, __u64 addr, __u64 size)
{
    // 检查是否启用详细事件
    __u32 key = 0;
    __u32 *verbose = bpf_map_lookup_elem(&config, &key);
    if (!verbose || !*verbose)
        return;

    struct alloc_event *e;

    e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);
    if (!e)
        return;

    __u64 pid_tgid = bpf_get_current_pid_tgid();
    e->pid = pid_tgid >> 32;
    e->tid = (__u32)pid_tgid;
    e->addr = addr;
    e->size = size;
    e->timestamp = bpf_ktime_get_ns();
    e->type = type;

    bpf_get_current_comm(&e->comm, sizeof(e->comm));

    bpf_ringbuf_submit(e, 0);
}
```

05 实战-追踪malloc操作的实现

■ 以用为本 拥抱创新

关键代码-用户空间核心代码

```
// 加载 BPF 程序
printf("Loading BPF program...\n");
obj = bpf_object__open_file("src/malloc_tracer.bpf.o", NULL);
if (!obj) {
    fprintf(stderr, "Failed to open BPF object\n");
    return 1;
}

err = bpf_object__load(obj);
if (err) {
    fprintf(stderr, "Failed to load BPF object: %d\n", err);
    bpf_object__close(obj);
    return 1;
}

// 配置详细模式
struct bpf_map *config_map = bpf_object__find_map_by_name(obj, "config");
if (config_map) {
    int config_fd = bpf_map__fd(config_map);
    __u32 config_key = 0;
    __u32 config_value = verbose ? 1 : 0;
    bpf_map_update_elem(config_fd, &config_key, &config_value, BPF_ANY);
}
```

```
// Ring Buffer 回调
static int handle_event(void *ctx, void *data, size_t data_sz)
{
    const struct alloc_event *e = data;
    const char *event_type;
    char size_str[32];

    switch (e->type) {
        case EVENT_MALLOC: event_type = "malloc"; break;
        case EVENT_FREE: event_type = "free"; break;
        case EVENT_CALLOC: event_type = "calloc"; break;
        case EVENT_REALLOC: event_type = "realloc"; break;
        default: event_type = "unknown"; break;
    }

    format_bytes(size_str, sizeof(size_str), e->size);

    printf("%-8llu %-16s %-8u %-8u %-8s %-16llx %-10s\n",
        e->timestamp / 1000000000, // 转换为秒
        e->comm,
        e->pid,
        e->tid,
        event_type,
        e->addr,
        size_str);

    return 0;
}
```

```
static void print_stats(int stats_fd)
{
    struct process_stats st;
    __u32 pid, next_pid;
    char comm[256];
    char total_str[32], current_str[32], peak_str[32];
    int has_next = 0;

    printf("\n=== Memory Allocation Statistics ===\n");
    printf("%-8s %-16s %-10s %-10s %-12s %-12s\n",
        "PID", "COMM", "ALLOCs", "FREES", "TOTAL", "CURRENT", "PEAK");
    printf("-----\n");

    while (bpf_map_get_next_key(stats_fd, has_next ? &pid : NULL, &next_pid) == 0) {
        pid = next_pid;
        has_next = 1;

        if (bpf_map_lookup_elem(stats_fd, &pid, &st) != 0)
            continue;

        // 尝试获取进程名称
        snprintf(comm, sizeof(comm), "/proc/%u/comm", pid);
        FILE *fp = fopen(comm, "r");
        if (fp) {
            if (fgetc(comm, sizeof(comm), fp))
                comm[strlen(comm) - 1] = 0;
            fclose(fp);
        } else {
            snprintf(comm, sizeof(comm), "<unknown>");
        }

        format_bytes(total_str, sizeof(total_str), st.total_bytes);
        format_bytes(current_str, sizeof(current_str), st.current_bytes);
        format_bytes(peak_str, sizeof(peak_str), st.peak_bytes);

        printf("%-8u %-16s %-10llu %-10llu %-12s %-12s\n",
            pid, comm,
            st.total_allocs,
            st.total_frees,
            total_str,
            current_str,
            peak_str);
    }
}
```

```
// 解析符号并附加探针
printf("Resolving symbols and attaching probes...\n");

struct {
    const char *name;
    const char *prog_enter_name;
    const char *prog_exit_name;
    bool has_retprobe;
} funcs[] = {
    {"malloc", "trace_malloc_enter", "trace_malloc_exit", true},
    {"free", "trace_free_enter", NULL, false},
    {"calloc", "trace_calloc_enter", "trace_calloc_exit", true},
};

for (int i = 0; i < sizeof(funcs) / sizeof(funcs[0]); i++) {
    size_t offset = get_symbol_offset_simple(libc_path, funcs[i].name);
    if (!offset) {
        fprintf(stderr, "Warning: Failed to find %s\n", funcs[i].name);
        continue;
    }

    printf(" %s @ 0x%lx\n", funcs[i].name, offset);

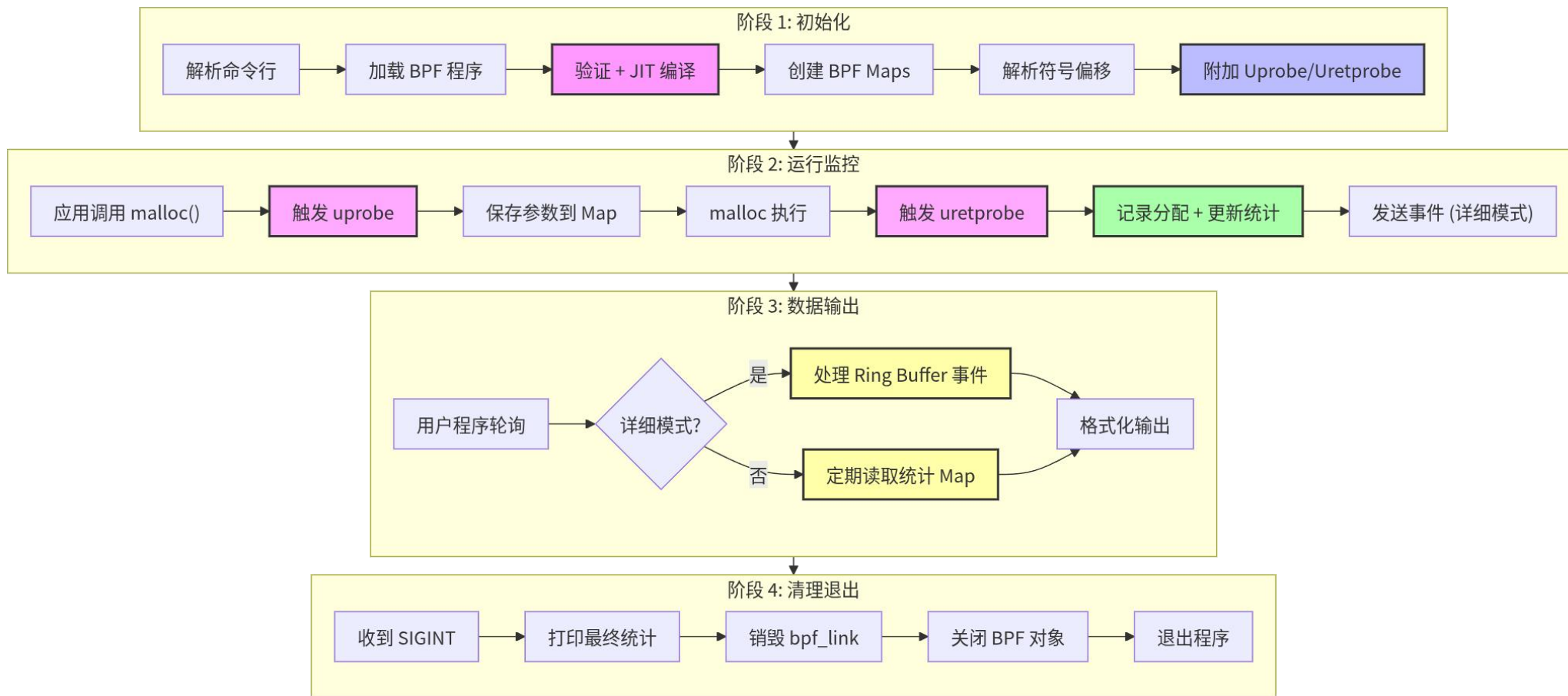
    // 附加 uprobe
    if (funcs[i].prog_enter_name) {
        prog = bpf_object__find_program_by_name(obj, funcs[i].prog_enter_name);
        if (prog) {
            links[link_count] = bpf_program__attach_uprobe(
                prog, false, -1, libc_path, offset);
            if (!links[link_count] || libbpf_get_error(links[link_count])) {
                fprintf(stderr, "Failed to attach uprobe to %s\n", funcs[i].name);
                goto cleanup;
            }
            link_count++;
        }
    }

    // 附加 uretprobe
}
```

05 实战-追踪malloc操作的实现

以用为本 拥抱创新

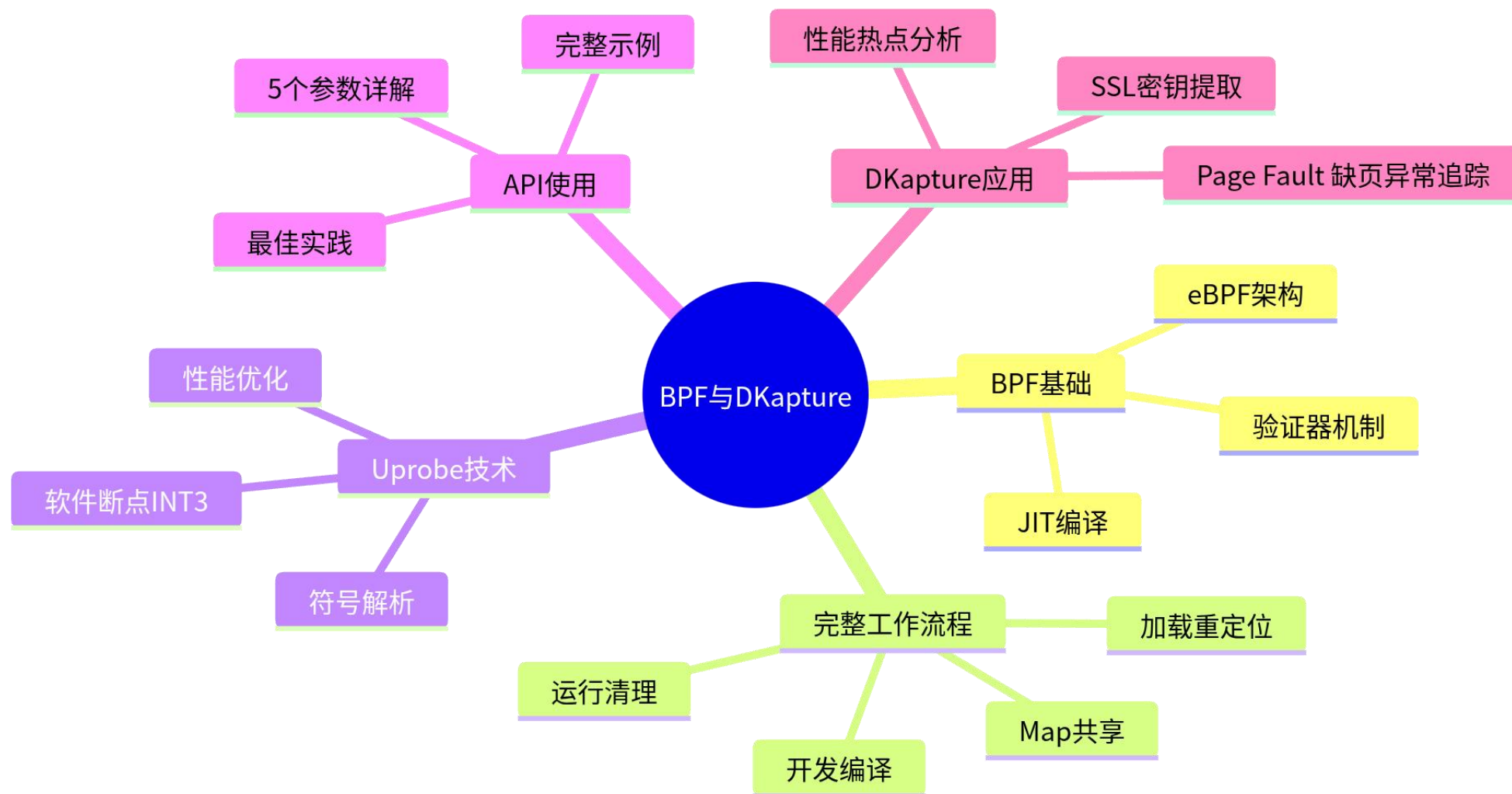
完整运行流程



06

总结





Thanks For Watching

深度科技社区



微信公众号

